UHM ECE 405 Spring 2026 Assignment 1: Basics

Anson Leung

February 22, 2026

1 Introduction

ECE405 Assignment 1
Repo @: https://github.com/Anson2Leung/ece405-assignment1-basics

2 Byte-Pair Encoding (BPE) Tokenizer

2.1 The Unicode Standard

Problem (unicode1): Understanding Unicode (1 point)

(a) What Unicode character does `chr(0)` return?
`'\x00'`, which represents the null character.

(b) How does this character's string representation `__repr__()` differ from its printed representation?
Printing out chr(0) results in an empty space (null character), which is shown in unicode as `'\x00'` (line 2 of Listing 1)

(c) What happens when this character occurs in text?
The null appears identical to a space, and are not used to represent the end of a line like in C.

```
chr(0) # '\x00'
print(chr(0)) #
"this is a test" + chr(0) + "string" # 'this is a test\x00string'
print("this is a test" + chr(0) + "string") # "this is a test" + chr(0) + "string"
len("this is a test" + chr(0) + "string") # 21
len("this is a test string") # 21
```

2.1.1

2.2 2.2 Unicode Encodings

Problem (unicode2): Unicode Encodings (3 points)

(a) What are some reasons to prefer training our tokenizer on UTF-8 encoded bytes, rather than UTF-16 or UTF-32? It may be helpful to compare the output of these encodings for various input strings.

```
ts1 = "hello! こんにちは!"
ts2 = "hello! 你好!"
ts3 = "abc ;.,"
test = [ts1, ts2, ts3]
enc_8 = [s.encode("utf-8") for s in test]
enc_16 = [s.encode("utf-16") for s in test]
enc_32 = [s.encode("utf-32") for s in test]
```

```python

for i, s in enumerate(test):
    print(f"--- '{s}' ---")
    print(len(enc_8[i]), enc_8[i])
    print(len(enc_16[i]), enc_16[i])
    print(len(enc_32[i]), enc_32[i])
```

2.2a code

```
--- 'hello! こんにちは!' ---
23 b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
28 b'\xff\xfeh\x00e\x00l\x00l\x00o\x00!\x00 \x00S0\x930k0a0o0!\x00'
56 b'\xff\xfe\x00\x00h\x00\x00\x00e\x00\x00\x00l\x00\x00\x00l\x00\x00\x00o\x00\x00
\x00!\x00\x00\x00 \x00\x00\x00S0\x00\x00\x930\x00\x00k0\x00\x00a0\x00\x00o0\x00\x00
!\x00\x00\x00'
--- 'hello! 你好!' ---
14 b'hello! \xe4\xbd\xa0\xe5\xa5\xbd!'
22 b'\xff\xfeh\x00e\x00l\x00l\x00o\x00!\x00 \x00`O}Y!\x00'
44 b'\xff\xfe\x00\x00h\x00\x00\x00e\x00\x00\x00l\x00\x00\x00l\x00\x00\x00o\x00\x00
\x00!\x00\x00\x00 \x00\x00\x00`O\x00\x00}Y\x00\x00!\x00\x00\x00'
--- 'abc ;.,' ---
7 b'abc ;.,'
16 b'\xff\xfea\x00b\x00c\x00 \x00;\x00.\x00,\x00'
32 b'\xff\xfe\x00\x00a\x00\x00\x00b\x00\x00\x00c\x00\x00\x00 \x00\x00\x00;\x00\x00
\x00.\x00\x00\x00,\x00\x00\x00'
```

2.2a output

Using UTF-8 encoding may be preferred to cut down on the space required to store the vocabulary. As observed with the 2.2a outputs the required length of the encoding significantly increases with non-ascii characters.

(b) Consider the following (incorrect) function, which is intended to decode a UTF-8 byte string into a Unicode string. Why is this function incorrect? Provide an example of an input byte string that yields incorrect results.

```python
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

print(decode_utf8_bytes_to_str_wrong("hello".encode("utf-8")))
print("hello! 你好!".encode("utf-8"))
print("hello! 你好!".encode("utf-8").decode("utf-8"))
print(decode_utf8_bytes_to_str_wrong("hello! 你好!".encode("utf-8"))) # error
```

2.2b code

```
hello
b'hello! \xe4\xbd\xa0\xe5\xa5\xbd!'
hello! 你好!
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe4 in position 0: unexpected
 end of data
```

2.2b output 1

The incorrect function goes byte-by-byte and decoding each one with UTF-8. So the Chinese example in the above blocks encodes to `\xe4\xbd\xa0\xe5\xa5\xbd`. The two Chinese characters, both start with the hex `e` or the binary `1110` indicating the Unicode is three bytes long, but from within the function the decode function is provided only one byte [**?**].

(c) Give a two byte sequence that does not decode to any Unicode character(s).

An example of a two byte sequence that does not decode to any Unicode characters is `\xff\x00` which fails for having a invalid start byte.

2.3 2.5 Experimenting with BPE Tokenizer Training

Problem (train_bpe): BPE Tokenizer Training (15 points)

See \assignment_files\bpe_tokenizer.py

Problem (train_bpe_tinystories): BPE Training on TinyStories (2 points)

(a) Train a byte-level BPE tokenizer on the TinyStories dataset, using a maximum vocabulary size of 10,000. Make sure to add the TinyStories `<|endoftext|>` special token to the vocabulary. Serialize the resulting vocabulary and merges to disk for further inspection. How many hours and memory did training take? What is the longest token in the vocabulary? Does it make sense?

See `assignment_colab.ipynb` for runner code, and refer to the output below.

The entire process took 426.08s and 64.82 MB, with the majority of the time being used for pretokenizaiton. It does make sense as the total amount of tasks to be completed is 1, so the chunking did not find a split causing the pretokenizing to be done by a single process instead of the two on Colab.

The longest token is "accomplishment", I believe that it makes sense as in the validation set there are 14 occurrences of accomplishment.

(b) Profile your code. What part of the tokenizer training process takes the most time? The majority of the time was spent on pre-tokenization, which includes chunking and encoding to utf-8.

```
Dataset:  /content/data/TinyStoriesV2-GPT4-train.txt
Number of processes requested:  2
Special tokens as bytes:  [b'<|endoftext|>']
Total number of tasks to complete:  1
Pre-tokenization complete. Unique pre-tokens: 47063
Number of merges to perform:  9743
Merging Vocab: 100%|[][][][][]| 9743/9743 [00:39<00:00, 244.60it/s]
Pre-tokenize time complete in 385.78s
Initialization time complete in 0.46s
BPE Merges complete in 39.84s
Total training time: 426.08s
Total memory used: 64.82 MB
--- Analysis Results ---
Longest Token ID: 7107
Longest Token Text: ' accomplishment'
Character Length: 15
```

2.5.1 output Colab

```
Dataset:  ..\..\data\TinyStoriesV2-GPT4-train.txt
Number of processes requested:  16
Special tokens as bytes:  [b'<|endoftext|>']
Total number of tasks to complete:  31
Pre-tokenization complete. Unique pre-tokens: 58144
Number of merges to perform:  9743
Merging Vocab: 100%|[][][][][]| 9743/9743 [00:50<00:00, 193.99it/s]
Pre-tokenize time complete in 118.29s
Initialization time complete in 0.56s
BPE Merges complete in 50.24s
Total training time: 169.09s
Total memory used: 117.75 MB
Vocab saved to TinyStoriesVocab.json
Merges saved to TinyStoriesMerges.json
--- Analysis Results ---
Longest Token ID: 7153
Longest Token Text: ' accomplishment'
Character Length: 15
```

2.5.1 output local

Problem (train_bpe _exots _owt): BPE Training on OpenWebText (2 points)

(a) Train a byte-level BPE tokenizer on the OpenWebText dataset, using a maximum vocabulary size of 32,000. Serialize the resulting vocabulary and merges to disk for further inspection. What is the longest token in the vocabulary? Does it make sense?

The longest token was 64 '-' characters in a row, which was surprising, until I did a search in the validation dataset, which found a large amount of '-' characters.

```
OpenWebText using vocab_size:32000
Dataset:  ..\..\data\owt_train.txt
Number of processes requested:  16
Special tokens as bytes:  [b'<|endoftext|>']
Total number of tasks to complete:  31
Pre-tokenization complete. Unique pre-tokens: 2143892
Number of merges to perform:  31743
Merging Vocab: 100%|[][][][][]| 31743/31743 [1:09:40<00:00,  7.59it/s]
Pre-tokenize time complete in 116.19s
Initialization time complete in 28.63s
BPE Merges complete in 4180.15s
Total training time: 4324.97s
Total memory used: 3725.39 MB
Vocab saved to OWTVocab.json
Merges saved to OWTMerges.json
--- Analysis Results ---
Longest Token ID: 25967
Longest Token Text: '----------------------------------------------------------------'
Character Length: 64
```

2.5.2 output

(b) Compare and contrast the tokenizer that you get training on TinyStories versus OpenWebText.

Due to the increase in file size from 2GB to 11GB the time it takes to pre-tokenize has increased, but the considerable increase is in the time it takes to merge due to the increased vocab size.

After observing some of the merged tokens the OpenWebText has more complex terms and proper nouns.

2.4 2.7 Experiments

Problem (tokenizer_experiments): Experiments with tokenizers (4 points)

(a) Sample 10 documents from TinyStories and OpenWebText. Using your previously-trained TinyStories and OpenWebText tokenizers (10K and 32K vocabulary size, respectively), encode these sampled documents into integer IDs. What is each tokenizer's compression ratio (bytes/token)?

Code in `assignment_colab.ipynb`. Output shown in `2.7.1 output`.

```
[TinyStories Tokenizer on TinyStories samples]
  Compression Ratio: 4.15 bytes/token
  Throughput:        1.45 MB/s
[OWT Tokenizer on OWT Samples]
  Compression Ratio: 4.69 bytes/token
  Throughput:        0.89 MB/s
[TinyStories Tokenizer on OWT Samples]
  Compression Ratio: 3.19 bytes/token
  Throughput:        0.31 MB/s

[The Pile Estimation]
```

```
Time to tokenize 825GB: 256.55 hours
```

2.7.1 output

(b) What happens if you tokenize your OpenWebText sample with the TinyStories tokenizer? Compare the compression ratio and/or qualitatively describe what happens.

Output shown in `2.7.1 output`. Using the TinyStories tokenizer on the OpenWebText sample, the compression rate is noticeably lower than utilizing OWT tokenizer on OWT samples. Since the vocabulary being found in OWT is more diverse the smaller vocabulary of TinyStories can not compress the tokens found in OWT as well.

(c) Estimate the throughput of your tokenizer (e.g., in bytes/second). How long would it take to tokenize the Pile dataset (825GB of text)?
Using the fastest throughput (1.45MB/s), it will take about 568966 seconds (158 hours) to tokenize 825000 MB of data. Using the OWT throughput (0.89 MB/s), it will take about 926966 seconds (257 hours).

(d) Using your TinyStories and OpenWebText tokenizers, encode the respective training and development datasets into a sequence of integer token IDs. We'll use this later to train our language model. We recommend serializing the token IDs as a NumPy array of datatype uint16. Why is uint16 an appropriate choice?

Utilizing uint16 is a good fit for token IDs as the max value for uint16 is $2^{16} - 1$ (65,535), meaning that it will adequately fit the 32k token vocabulary.

```
Calculating boundaries for ../data/TinyStoriesV2-GPT4-train.txt...
Encoding 159 chunks using 40 processes...
100%|[][][][][]| 159/159 [11:48<00:00,  4.45s/it]
Finalizing 537848181 tokens...
Completed! Saved to assignment_files/tinystories_train.npy
Final size: 1025.86 MB

Calculating boundaries for ../data/owt_train.txt...
Encoding 159 chunks using 40 processes...
100%|[][][][][]| 159/159 [1:05:44<00:00, 24.81s/it]
Finalizing 1810995702 tokens...
Completed! Saved to ../data/owt_train.npy
Final size: 3454.20 MB
```

2.7.2 output

3 Transformer Language Model Architecture

3.1 3.6 The Full Transformer LM

Problem Problem (transformer_accounting): Transformer LM resource accounting (5 points)

```
vocab_size : 50,257
context_length : 1,024
num_layers : 48
d_model : 1,600
num_heads : 25
d_ff : 6,400
```

3.6.1 Configuration

Suppose we constructed our model using this configuration. How many trainable parameters would our model have? Assuming each parameter is represented using single-precision floating point, how much memory is required to just load this model?

```
Parameters for token embedding: vocab_size * d_model
Parameters for positional embedding: context_length * d_model
Weight parameters for MHSA (Query Key Value and output projections): d_model * d_model
    * 3 + d_model * d_model
SwiGLU feed forward (Linear): (d_ff * d_model) * 2
RMSNorm: d_model * 2
Transformer block = SwiGLU FF + MHSA + RMSNorm
Total parameters = embeddings + Transfomerblock * num_layers

token embeddings = 50257 * 1600
positional embeddings = 1024 * 1600
MHSA = 4*1600^2
SwiGLU = 6400 * 1600 * 2
RMSNorm = 1600 * 2
Transformer block = 6400 * 1600 * 2 + 4*1600^2 + 1600 * 2
Total = 50257 * 1600 + 1024 * 1600 + 30723200 * 48
Total = 1.5567632*10^{9}
```

3.6.1 Parameter calculations

Total number of parameters is 1.5567632×10^9. If each parameter is represented by a single-precision floating point (4 bytes). The total amount of memory to hold all of the parameters is 6.2270528×10^9 bytes or 6.23 Gigabytes or 5.80 Gibibytes.

(b)Identify the matrix multiplies required to complete a forward pass of our GPT_2 XL-shaped model. How many FLOPs do these matrix multiplies require in total? Assume that our input sequence has context_length tokens.

C = context_length
D = d_model
F = d_ff
V = vocab_size
MHSA matrix multiplications

- QKV Projections $2 \times (C, D) \times (D, 3D) = 2 \times 3CD^2$
- Attention Scores $2 \times (C, D) \times (D, C) = 2 \times C^2D$
- Attention Context $2 \times (C, D) \times (D, C) = 2 \times C^2D$
- Output $2 \times (C, D) \times (D, D) = 2 \times CD^2$
- Total $8CD^2 + 4C^2D$

SwiGLU Feed Forward

- Gate Projection $2 \times (C, D) \times (D, F) = 2TDF$
- Up Projection $2 \times (C, D) \times (D, F) = 2TDF$
- Down Projection $2(C, F) \times (C, D) = 2CDF$
- Total $6CDF$

Final Transformation

- Projection $2 \times (C, D) \times (D, V)$
- Total $2CDV$

```python
vocab_size = 50257
context_length= 1024
num_layers = 48
d_model = 1600
num_heads = 25
d_ff = 6400
C = context_length
D = d_model
F = d_ff
V = vocab_size

def flopAnalysis(C,D,F,V, num_layers=1):
    f1=flopsMHSA(C, D, num_layers)
    f2=flopsFF(C, D, F, num_layers)
    f3=flopsFinal(C, D, V)
    total = f1+f2+f3
    print(f"MHSA:                 {f1:18,.0f} ({f1/total:>6.2%})")
    print(f"SwiGLU Feed Forward: {f2:18,.0f} ({f2/total:>6.2%})")
    print(f"Final Transformation:{f3:18,.0f} ({f3/total:>6.2%})")
    print(f"Total                 {total:18,.0f}")
flopAnalysis(C,D,F,V,num_layers)
```

3.6.2a code

```
MHSA:                  1,328,755,507,200 (29.44%)
SwiGLU Feed Forward:   3,019,898,880,000 (66.91%)
Final Transformation:    164,682,137,600 ( 3.65%)
Total                  4,513,336,524,800
```

3.6.2b Flops

(c) Based on your analysis above, which parts of the model require the most FLOPs?

SwiGLU's feed forward has the most amount of FLOPs at about 3 trillion.

(d) Repeat your analysis with GPT-2 small (12 layers, 768 d_model, 12 heads), GPT-2 medium (24layers, 1024 d_model, 16 heads), and GPT-2 large (36 layers, 1280 d_model, 20 heads). As the model size increases, which parts of the Transformer LM take up proportionally more or less of the total FLOPs?

```
----------Small----------
MHSA:                     96,636,764,160 (27.64%)
SwiGLU Feed Forward:     173,946,175,488 (49.75%)
Final Transformation:     79,047,426,048 (22.61%)
Total                    349,630,365,696
----------Medium----------
MHSA:                    309,237,645,312 (29.93%)
SwiGLU Feed Forward:     618,475,290,624 (59.87%)
Final Transformation:    105,396,568,064 (10.20%)
Total                  1,033,109,504,000
----------Large----------
MHSA:                    676,457,349,120 (29.96%)
SwiGLU Feed Forward:   1,449,551,462,400 (64.20%)
Final Transformation:    131,745,710,080 ( 5.84%)
Total                  2,257,754,521,600
----------XL----------
MHSA:                  1,328,755,507,200 (29.44%)
SwiGLU Feed Forward:   3,019,898,880,000 (66.91%)
Final Transformation:    164,682,137,600 ( 3.65%)
Total                  4,513,336,524,800
```

3.6.3 output

As the model size increases the Feed Forward component takes up an increasingly larger portion of FLOPs. MultiHeadSelfAttention also utilizes a considerably large number of FLOPs.

(e) Take GPT-2 XL and increase the context length to 16,384. How does the total FLOPs for one forward pass change? How do the relative contribution of FLOPs of the model components change?

```
----------1024----------
MHSA:                  1,328,755,507,200 (29.44%)
SwiGLU Feed Forward:  3,019,898,880,000 (66.91%)
Final Transformation:   164,682,137,600 ( 3.65%)
Total                  4,513,336,524,800
----------16384 (+1600.00%)----------
MHSA:                 98,569,499,443,200 (65.92%)
SwiGLU Feed Forward: 48,318,382,080,000 (32.32%)
Final Transformation: 2,634,914,201,600 ( 1.76%)
Total                 149,522,795,724,800
3312.91% increase in FLOPs
```

3.6.4 outputs

Increasing the context length to 16,384 results in a larger increase of FLOPs for the MHSA component. A 16x increase in context length resulted in a 33x increase in FLOPs.

4 Training a Transformer LM

4.1 4.2.1 Implementing SGD in PyTorch

Problem (learning_rate_tuning): Tuning the learning rate (1 point)

```
-----Learning rate 1-----
21.261045455932617
20.419109344482422
19.845653533935547
19.389982223510742
19.00412368774414
18.66568946838379
18.36212158203125
18.085561752319336
17.830698013305664
17.593748092651367
-----Learning rate 10.0-----
21.76651382446289
13.93056869506836
10.269020080566406
8.034412384033203
6.507874011993408
5.395773410797119
4.550618648529053
3.8886330127716064
3.3581409454345703
2.9253134727478027
-----Learning rate 100.0-----
26.923503875732422
26.923503875732422
4.619342803955078
0.11055128276348114
1.3117979401792648e-16
1.4620800931883836e-18
4.92333797000242e-20
2.93286660482836e-21
2.5160025636605025e-22
2.7955583339462557e-23
-----Learning rate 1000.0-----
24.47492218017578
8835.4453125
1526020.5
169753504.0
13750032384.0
867784523776.0
```

```
44549225316352.0
1916698009534464.0
7.064540879100314e+16
2.2685023573838397e+18
```

4.2.1 output

For the lower learning rates the loss decays to zero (getting to zero faster for learning rates of 10 or 100). But with the learning rate = 100 the loss diverges extremely with each step.

4.2 4.3 AdamW

(a) AdamW: An algebraic expression for each of parameters, activations, gradients, and optimizer state, as well as the total.

Vocabulary size (V), context length (C), batch size (B), number of layers (L), model dimension (D), d_ff ($4D$) feed forward (F), and number of heads (H).

Transformer (Multiply total by L layers) $L \times B \times (14CD + 2BC^2)$

- RMSNorm : $2 \times (B \times C \times D)$

  Transformer (MHSA) $= 5BCD + 2BC^2$
- QKV Projections: $B \times C \times D$
- Attention scores QTK Matrix Multiply: $B \times H \times C^2$
- Softmax: $B \times H \times C^2$
- Weighted Sum of Values: $B \times C \times D$
- Output Projection: $B \times C \times D$

  Transformer (FF) $= 9BCD$
- FF input W1 Matrix Multiply: $B \times C \times D$
- SiLU: $B \times C \times 4D$
- W2 Matrix Multiply: $B \times C \times 4D$

  Final $= B(CD + CV + C)$
- Final RMSNorm: $B \times C \times D$
- Output Embedding logits: $B \times C \times V$
- Cross-entropy: $B \times C$

  Grand Total $= B[L(14CD + 2HC^2) + CD + CV + C]$

Parameters (P) $= 4[(V \times D) + (C \times D) + L(12D^2 + 2D) + D]$
Gradient $= 4 \times P$
ADAMW Optimizer $= 2 \times (4 \times P) = 8 \times P$
Total $= 16P + 4B[L(11CD + 2HC^2) + CD + CV]$

(b) Instantiate your answer for a GPT-2 XL-shaped model to get an expression that only depends on the batch_size. What is the maximum batch size you can use and still fit within 80GB memory?.

```
V = 50257   # vocab_size
D = 1600    # d_model
C = 1024    # context_length
L = 48      # num_layers
H = 25      # num_heads
B=1
P = (V * D + C * D) + L * (12 * D**2 + 2 * D) + D
```

```python

static_bytes = 16 * P

act_per_unit_B =4*B*(L*(11*C*D + 2*H*C**2) + C*D + C*V)

LIMIT_GB = 80
limit_bytes = LIMIT_GB * (10**9) # Using GB

max_B = (limit_bytes - static_bytes) / act_per_unit_B

print(f"Total Parameters (P): {P:,}")
print(f"Static Memory (16P):  {static_bytes / 1e9:.2f} GB")
print(f"Activation per B:     {act_per_unit_B / 1e9:.2f} GB")
print(f"Max Batch Size (B):   {max_B//1}")
```

4.3.1a code

```
Total Parameters (P): 1,556,764,800
Static Memory (16P):  24.91 GB
Activation per B:     13.74 GB
Max Batch Size (B):   4.0
```

4.3.1b output

Maximum batch size was found to be 4.

(c) How many FLOPs does running one step of AdamW take?

```
First Moment:  b_1 * m + (1 - beta1) * g = 2 FLOPs
Second Moment: b_2 * v + (1 - b_2) * g^2 = 3 FLOPs
Bias correction: a * sqrt( 1-(b2)^t ) / ( 1-(b1)^t ) = 4 FLOPs
Update Parameters: theta - a_t * m/(sqrt(v)+eps) = 3 FLOPs
Apply Weight Decay: theta + (-lr * wd * theta) = 2 FLOPs

14 * parameters(1556764800) = 2.17947072*10^10
```

4.3.2

A total of 14 FLOPs per parameter for one step of AdamW. For a total of $2.17947072 \times 10^{10}$ FLOPs

(d) Model FLOPs utilization (MFU) is defined as the ratio of observed throughput (tokens per second) relative to the hardware's theoretical peak FLOP throughput

```python
P = 1556764800 # parameters
B = 1024 # Batch size
C = 1024 # context length
steps = 400000

tokens_per_step = B * C
flops_per_step = 6 * P * tokens_per_step # Forward (2P) and backward pass (4P)
total_flops = flops_per_step * steps

# Hardware A100
peak_flops = 19.5e12 # 19.5 tera flops / second
mfu  = 0.50 # 50%
effective_flops = peak_flops * mfu # effective FLOPs / sec

seconds = total_flops / effective_flops
days = seconds / 86400

print(days)
```

4.3.3 code

This results in 4650.673 days, so about 4651 days. So training on a single GPU is infeasible and will require hundreds of GPUs to train the model in a reasonable time.

5 Training loop

All code section.

6 Generating text

All code section.

7 Experiments

7.1 How to Run Experiments and Deliverables

7.1.1 Problem (experiment_log): Experiment logging

For your training and evaluation code, create experiment tracking infrastructure that allows you to track your experiments and loss curves with respect to gradient steps and wallclock time.

Example log output

```
2026-02-20 03:47:07 | INFO | step   1720 | loss 2.3188 | ppl 10.16 | lr 8.46e-05 |
    elapsed 990.8s | ms/step 576.0 | tokens 35,225,600
```

7.1.1

7.2 TinyStories

7.2.1 Problem (learning_rate): Tune the learning rate

(a)hyperparameter sweep over the learning rates and report the final losses (or note divergence if the optimizer diverges).

Using a max iterations/ steps of 5000 for these sweeps to find a good LR. If it does not fall under 1.45 then I can sweep again or increase the amount of steps.

```
# Filepath locations
TOK_TRAIN="/home/ansonl32/ECE405/data/tinystories_train.npy"
TOK_VALID="/home/ansonl32/ECE405/data/tinystories_valid.npy"

cd /home/ansonl32/ECE405/ece405-assignment1-basics

# TODO switch out LR values depending on best result
for LR in 3e-5 1e-4 3e-4 6e-4 1e-3; do
    echo "========================================"
    echo "Starting run with LR=$LR"
    echo "========================================"
    uv run python -m assignment_files.TrainLoop \
        --train_data      "$TOK_TRAIN" \
        --val_data        "$TOK_VALID" \
        --vocab_size      10000 \
        --d_model         512 \
        --context_length  256 \
        --num_layers      4 \
        --num_heads       16 \
        --d_ff            1344 \
        --theta           10000 \
        --batch_size      80 \
        --max_steps       5000 \
        --warmup_steps    500 \
        --lr              "$LR" \
        --beta1           0.9 \
        --beta2           0.999 \
        --weight_decay    0.1 \
        --grad_clip       1.0 \
        --min_lr_ratio    0.1 \
```

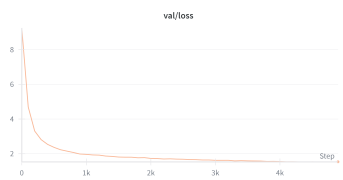
```
        --log_every       20 \
        --val_every       100 \
        --val_steps       20 \
        --checkpoint_every 5000 \
        --checkpoint_dir  /home/ansonl32/koa_scratch/ECE405/assignment1-basics/checkpoints/lr_${LR} \
        --wandb_project   ece405 \
        --wandb_run_name  "a_lr_${LR}"
done
```

7.2.1a sweep1

```

3e-5: val/loss: 2.611782956123352
1e-4: RuntimeError: CUDA error: device-side assert triggered
3e-4: val/loss: 1.687470406293869
6e-4: val/loss: 1.577734011411667
1e-3: RuntimeError: CUDA error: device-side assert triggered

9e-4: 1.5341815948486328
1e-3: 1.4979049384593963
5e-3: 1.452186369895935
1e-2: Diverge
```

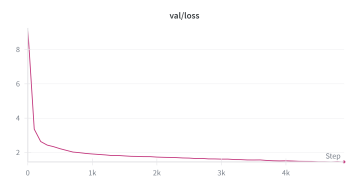
7.2.1a results

After the first batch of runs I continued exploring until it stopped (tried 1e-2 twice). Could continue to fine-tune some more between the two, but will take too much time.



(a) LR9e-4

(b) LR1e-3



(c) LR5e-3

(b)Folk wisdom is that the best learning rate is "at the edge of stability." Investigate how the point at which learning rates diverge is related to your best learning rate.

Figure 2: LR1e-2

The two extremes of learning rates is going too slow, or never making it there (divergence). In SGD

a learning rate that is a little too high will cause jumps that overshoot a minima, but it is possible that it will find its way as the LR decays.

The loss values appear to follow the same trend as the other previous learning rates, but would ended up failing.

7.2.2 Problem (batch_size_experiment): Batch size variations

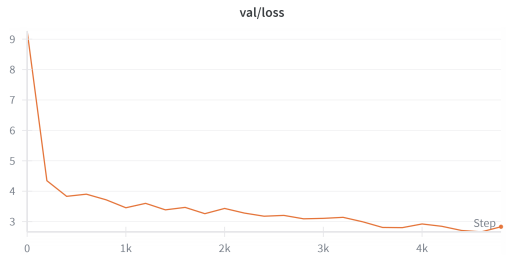
Vary your batch size all the way from 1 to the GPU memory limit. Try at least a few batch sizes in between, including typical sizes like 64 and 128.

```
uv run python -m assignment_files.TrainLoop \
       --train_data      "$TOK_TRAIN" \
       --val_data        "$TOK_VALID" \
       --vocab_size      10000 \
       --d_model         512 \
       --context_length  256 \
       --num_layers      4 \
       --num_heads       16 \
       --d_ff            1344 \
       --theta           10000 \
       --batch_size      "$BATCH_SIZE" \
       --max_steps       5000 \
       --warmup_steps    500 \
       --lr              "$BEST_LR" \
       --beta1           0.9 \
       --beta2           0.999 \
       --weight_decay    0.1 \
       --grad_clip       1.0 \
       --min_lr_ratio    0.1 \
       --log_every       100 \
       --val_every       200 \
       --val_steps       20 \
       --checkpoint_every 5000 \
       --checkpoint_dir  /home/ansonl32/koa_scratch/ECE405/assignment1-basics/checkpoints/TSbatch_${BATCH_SIZE} \
       --wandb_project   ece405 \
       --wandb_run_name  "TSbatch_${BATCH_SIZE}_lr_${BEST_LR}"
```

7.2.2a settings

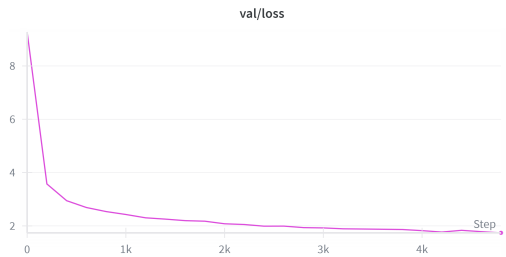
```
Trying batch sizes on ECE partition
iterations = 5000, lr=1e-3
32: Out of memory
64: Failed
128: Out of memory
256: Out of memory

1: 2.831978988647461
8: 1.9416767239570616
16: 1.738572520017624
48: 1.5689158022403715
```

7.2.2b results

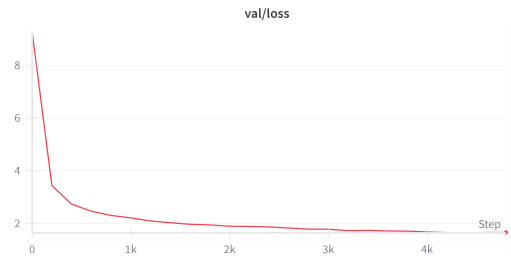


(a) LR 1e-3 BS1

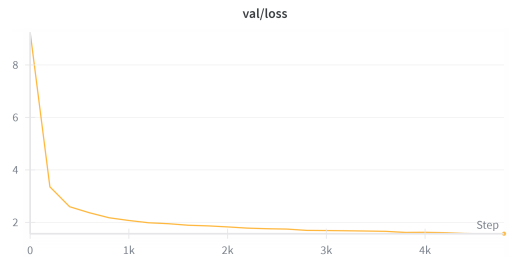
(b) LR 1e-3 BS 8



(c) LR 1e-3 BS16



(d) LR 1e-3 BS 32



(e) LR 1e-3 BS 48

As the batch size increased the loss decreased, the time needed to train increased. And going even higher will easily cause the GPU to run out of memory.

7.2.3 Problem (generate): Generate text

Using your decoder and your trained checkpoint, report the text generated by your model. You may need to manipulate decoder parameters (temperature, top-p, etc.) to get fluent outputs.

```
uv run python -m assignment_files.GenerateText \
    --checkpoint /home/ansonl32/koa_scratch/ECE405/assignment1-basics/checkpoints/lr_1e-3/ckpt_final.pt \
    --tokenizer_vocab  /home/ansonl32/ECE405/ece405-assignment1-basics/assignment_files/TinyStoriesVocab.json \
    --tokenizer_merges /home/ansonl32/ECE405/ece405-assignment1-basics/assignment_files/TinyStoriesMerges.json \
    --prompt          "Once upon a time" \
    --max_new_tokens  256 \
    --temperature     0.8 \
    --top_p           0.9 \
    --vocab_size      10000 \
    --device          cuda:0
```

7.2.3a settings

```
LR = 1e-3, Val/Loss = 1.497, batchsize = 80, iterations= 5000
Note that I added newlines to support formatting
------
Once upon a time, there was a little girl named Amy. Amy had a pet cat named Fluffy.
Fluffy was a poor cat who loved to eat and sleep all day. Amy loved Fluffy very much.
One day, Amy and Fluffy went for a walk in the park.
They saw a big tree and decided to climb it.
Amy was so happy to be there. She started to climb the tree.
Fluffy was excited and wanted to climb higher and higher.
As Fluffy climbed higher, she saw a bird's nest with a nest.
Fluffy was surprised and asked the bird, "Why are you so high up there?"
The bird said, "I am very high and I need help. Can you help me get the nest?"
Fluffy wanted to help, so he had an idea.
Fluffy climbed up the tree and got the nest for the bird.
The bird was so happy and thanked Fluffy.
They became good friends and played together every day.
And from that day on, they always helped each other when they needed it.
```

7.2.3b output

Link to checkpoint on Koa: https://koa.its.hawaii.edu/node/gn-15-35-00.mana.ci.its.hawaii.edu/8883/lab/tree/ECE405/ece405-assignment1-basics/assignment_files/models/lr_1e-3

The generated output appears coherent, but is lacking consistency. Such as the Fluffy being a poor cat (lazy cat). Amy was climbing the tree, then it jumps to Fluffy wanting to climb higher. Then it has "Bird's nest with a nest" and helping a bird retrieve a nest.

The initial prompt has a strong effect on the generation (was easier to see on OWT). Since TS is aimed at children the prompt of "Once upon a time" is a good start to many fairy tales. Having a good loss score is also indicative of having a coherent generation.

7.3 Ablations and architecture modification

7.3.1 Problem (layer_norm_ablation): Remove RMSNorm and train

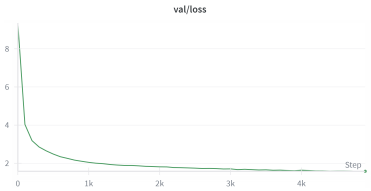
Remove all of the RMSNorms from your Transformer and train. What happens at the previous optimal learning rate? Can you get stability by using a lower learning rate?

```
for LR in 3e-4 6e-4 1e-3; do
    echo "========================================"
    echo "Starting run with LR=$LR"
    echo "========================================"
    uv run python -m assignment_files.TrainLoop \
        --train_data      "$TOK_TRAIN" \
        --val_data        "$TOK_VALID" \
        --vocab_size      10000 \
        --d_model         512 \
        --context_length  256 \
        --num_layers      4 \
        --num_heads       16 \
        --d_ff            1344 \
        --theta           10000 \
        --batch_size      80 \
        --max_steps       5000 \
        --warmup_steps    500 \
        --lr              "$LR" \
        --beta1           0.9 \
        --beta2           0.999 \
        --weight_decay    0.1 \
        --grad_clip       1.0 \
        --min_lr_ratio    0.1 \
        --log_every       20 \
        --val_every       100 \
        --val_steps       20 \
        --checkpoint_every 5000 \
        --checkpoint_dir  /home/ansonl32/koa_scratch/ECE405/assignment1-basics/checkpoints/nonorm_lr_${LR} \
        --wandb_project   ece405 \
        --wandb_run_name  "nonorm_lr_${LR}" \
```

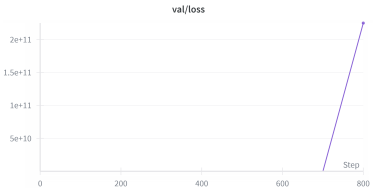
```
        --ablation        no_norm
done
```

7.3.1 settings

(a) LR 3e-4 (1.7435854077339172)



(b) LR 6e-4 (1.5816202402114867)

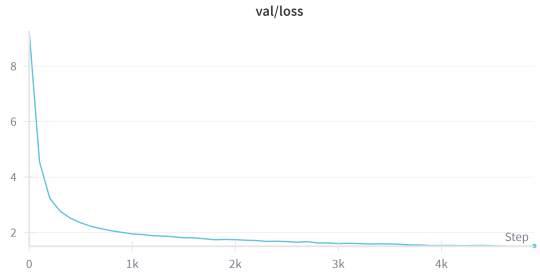


(c) LR 1e-3 (Diverged)

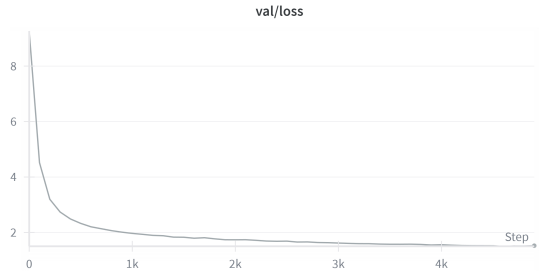
LR: 3e-4 Crashed at step 4220 With the previous best Learning rate of 1e-3 for the no normalization model it ends up diverging. Now a lower learning rate such as 6e-4 appears to be doing slightly worse than before, but it does not diverge like 1e-3. So including RMSNorm helps the model converge as seen with 1e-3.

7.3.2 Problem (pre_norm_ablation): Implement post-norm and train

Modify your pre-norm Transformer implementation into a post-norm one. Train with the post-norm model and see what happens



(a) Prenorm Loss (1.5195569574832917)



(b) Postnorm Loss (1.620127761363983)

Figure 5

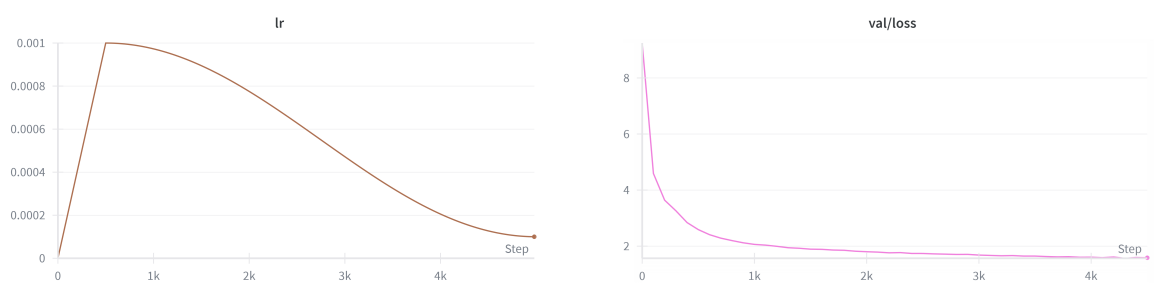
Both are using LR 1e-3, the curves look similar, but the Loss for the Postnorm model is slightly higher than the Prenorm model.

7.3.3 Problem (no_pos_emb): Implement NoPE

Modify your Transformer implementation with RoPE to remove the position embedding information entirely, and see what happens.

```
for LR in 1e-3 9e-4
    for ABLATION in none nope; do
        echo "========================================"
        echo "Ablation: $ABLATION | LR=$LR"
        echo "========================================"
        uv run python -m assignment_files.TrainLoop \
            --train_data      "$TOK_TRAIN" \
            --val_data        "$TOK_VALID" \
            --vocab_size      10000 \
            --d_model         512 \
            --context_length  256 \
            --num_layers      4 \
            --num_heads       16 \
            --d_ff            1344 \
            --theta           10000 \
            --batch_size      80 \
            --max_steps       5000 \
            --warmup_steps    400 \
            --lr              "$LR" \
            --beta1           0.9 \
            --beta2           0.999 \
            --weight_decay    0.1 \
            --grad_clip       1.0 \
            --min_lr_ratio    0.1 \
            --log_every       20 \
            --val_every       100 \
            --val_steps       20 \
            --checkpoint_every 4000 \
            --checkpoint_dir  /home/ansonl32/koa_scratch/ECE405/assignment1-basics/checkpoints/nope_${ABLATION} \
            --wandb_project   ece405 \
            --wandb_run_name  "nope_${ABLATION}_$LR" \
            --ablation        "$ABLATION"
    done
done
```

7.3.3 settings



(a) RoPE 1e-3 Loss (1.4979049384593963)

(b) NoPE 1e-3 failed (1.5840522706508635)

Figure 6: RoPE vs NoPE

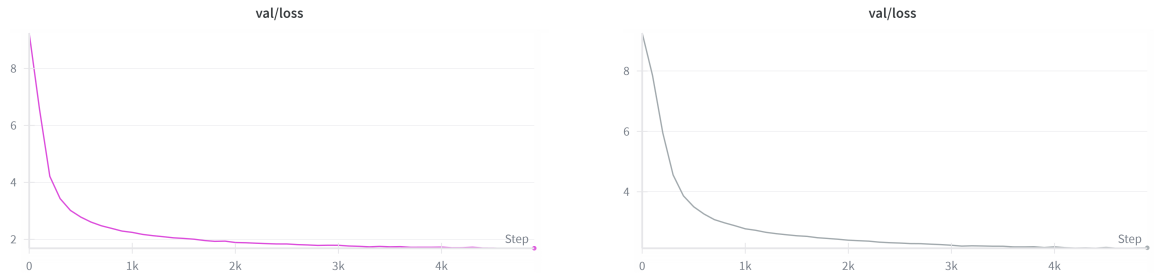
Nope run failed at 4500 steps with a final loss of 1.584. Which is a substancial bit higher than the Rope version.

7.3.4 Problem (swiglu_ablation): SwiGLU vs. SiLU)

```
uv run python -m assignment_files.TrainLoop \
        --train_data      "$TOK_TRAIN" \
        --val_data        "$TOK_VALID" \
        --vocab_size      10000 \
        --d_model         512 \
        --context_length  256 \
        --num_layers      4 \
        --num_heads       16 \
        --d_ff            1344 \
```

```
        --theta           10000 \
        --batch_size      80 \
        --max_steps       5000 \
        --warmup_steps    500 \
        --lr              1e-4 \
        --beta1           0.9 \
        --beta2           0.999 \
        --weight_decay    0.1 \
        --grad_clip       1.0 \
        --min_lr_ratio    0.1 \
        --log_every       20 \
        --val_every       100 \
        --val_steps       20 \
        --checkpoint_every 5000 \
        --checkpoint_dir  /home/ansonl32/koa_scratch/ECE405/assignment1-basics/
    checkpoints/swiglu_${ABLATION} \
        --wandb_project   ece405 \
        --wandb_run_name  "swiglu_${ABLATION}_lr3e-4" \
        --ablation        "$ABLATION"
```

7.3.4 settings



(a) SwiFLU Loss 1.687470406293869

(b) SiLU Loss (2.2293022871017456)

Figure 7: SwiGLU vs SiLU

The validation loss when using SiLU instead of SwiGLU vs higher. It is to be noted that SwiGLU was using a 3e-4 LR while SiLU was utilizing a 1e-4 LR. I tried running on 3e-4 it ran into a device-side error.

7.4 Running on OpenWebText

7.4.1 Problem (main_experiment): Experiment on OWT

Train your language model on OpenWebText with the same model architecture and total training iterations as TinyStories. How well does this model do

```
Using max step 3000 for sweep
3e-4: val/loss:5.102694702148438
6e-4: val/loss:4.845708513259888
1e-3: val/loss:4.752266788482666
3e-3: val/loss:5.666153359413147
5e-3: val/loss:4.875535249710083
8e-3: val/loss:5.25123085975647
```

7.4.1 OWT LR

```
#SBATCH --job-name=ECE405_OWT_ks
#SBATCH --gres=gpu:1
#SBATCH --partition=kill-shared
#SBATCH --time=12:00:00
#SBATCH --cpus-per-task=16
#SBATCH --mem=256GB
```

```
#SBATCH --error=/home/ansonl32/koa_scratch/ECE405/assignment1-basics/logs/owt_error%j.log
#SBATCH --output=/home/ansonl32/koa_scratch/ECE405/assignment1-basics/logs/owt%j.log
export WANDB_API_KEY=""
# OWT tokenized data
TOK_TRAIN="/home/ansonl32/ECE405/data/owt_train.npy"
TOK_VALID="/home/ansonl32/ECE405/data/owt_valid.npy"
cd /home/ansonl32/ECE405/ece405-assignment1-basics
uv run python -m assignment_files.TrainLoop \
    --train_data      "$TOK_TRAIN" \
    --val_data        "$TOK_VALID" \
    --vocab_size      32000 \
    --d_model         512 \
    --context_length  256 \
    --num_layers      4 \
    --num_heads       16 \
    --d_ff            1344 \
    --theta           10000 \
    --batch_size      128 \
    --max_steps       10000 \
    --warmup_steps    1000 \
    --lr              1e-3 \
    --beta1           0.9 \
    --beta2           0.999 \
    --weight_decay    0.1 \
    --grad_clip       1.0 \
    --min_lr_ratio    0.1 \
    --log_every       100 \
    --val_every       200 \
    --val_steps       20 \
    --checkpoint_every 5000 \
    --checkpoint_dir  /home/ansonl32/koa_scratch/ECE405/assignment1-basics/checkpoints/owt_1e3_10000_kill \
    --wandb_project   ece405 \
    --wandb_run_name  "owt_lr1e-3_10000_kill" \
    --ablation        none
```

7.4.2 sbatch config

```
uv run python -m assignment_files.GenerateText \
    --checkpoint /home/ansonl32/koa_scratch/ECE405/assignment1-basics/checkpoints/owt_1e3_10000_kill/ckpt_final.pt \
    --tokenizer_vocab  /home/ansonl32/ECE405/ece405-assignment1-basics/assignment_files/OWTVocab.json \
    --tokenizer_merges /home/ansonl32/ECE405/ece405-assignment1-basics/assignment_files/OWTMerges.json \
    --prompt          "My name is" \
    --max_new_tokens  256 \
    --temperature     0.8 \
    --top_p           0.9 \
    --vocab_size      32000 \
    --device          cuda:0
```

7.4.3a settings

```
Replaced apostrophies with single quotes
LR = 1e-3, val/loss 3.993, batchsize = 128, iterations = 10000
-----
My name is that of a typical professional classmate. I am sure that I am confident that I will, as a new school teacher. My second child is obviously not a type of kid I am going to have, but I am confident I will make my first child come to me.

What I am saying is that I am not prepared to step in and play in a world where I will not be able to move myself to the next phase of the future. I will not only be able to see myself, but I will try and force myself to not get into the next phase of the future. I am a happy life, but I am very grateful for the opportunity and experience.

So I am ready to play at this year's Women's and Women's Council in Wales. I will be looking forward to this. It will be my final year of the year and I will be
```

```
    determined to do the job.
```

7.4.3b output

Link to checkpoint on Koa: https://koa.its.hawaii.edu/node/gn-15-35-00.mana.ci.its.hawaii.edu/8883/lab/tree/ECE405/ece405-assignment1-basics/assignment_files/models/owt_1e3_10000_kill

The text is a lot more incoherent, being in between a cover letter, casual self-introduction, and congratulatory speech. This is due to a wide range of training material on the Web such as blogs posts, social media posts, news articles, that have different styles of writing and purpose. Which is quite different to the fairy tale style of TS.

Figure 8: LR1e-3